# CSE4102
# Programming Languages

Lecture 9: Judgments, Inference Rules and Lambda Calculus

# Announcements

- HW2 due tomorrow (not today)
- Office hours today, 1-2pm
- Lecture videos from Tuesday on HuskyCT – watch by 3/6
- Midterm 3/3 11-12:15am
  - Details sent by email on Tuesday

# Today

1. How PL researchers state things about PLs (inference rules, derivations)
2. How PL researchers represent PLs (core calculi)
3. History of the untyped lambda calculus
4. Syntax of the untyped lambda calculus

# We state things with *judgments*

logical statements, properties of programs, math equations, …

- Judgment: statement with variables (*metavariables*) standing in for other things/expressions

Important: the parts in blue don't change

- Examples:
  - X + Y = Z
    - 1 + 2 = 3
  - e is a natural number
    - 1 is a natural number
  - e : t ("e has type t")
    - "Hello" : string

2 + 2 = 5

3.5 is a natural number

1 + 2 : string

# We define what instances of a judgment hold with *inference rules*

$$\frac{\text{if everything above this line is true}}{\text{then we can conclude everything below the line}}$$

"premises"

(Rule Name)

"conclusion"

A particular form or transformation is defined by a set of rules:

$$\frac{n \text{ is a natural number}}{n + 1 \text{ is a natural number}}$$

A rule with no premises is called an *axiom*:

$$\frac{}{0 \text{ is a natural number}}$$

# We *derive* new facts by putting together rules from the system

$$\frac{}{0 \text{ is a natural number}}$$

$$\frac{n \text{ is a natural number}}{n+1 \text{ is a natural number}}$$

$$\frac{\dfrac{\dfrac{\dfrac{}{0 \text{ is a natural number}}}{1 \text{ is a natural number}}}{2 \text{ is a natural number}}}{3 \text{ is a natural number}}$$

"derivation"

# Another example

- X is XML

$$\frac{}{\texttt{""}\ \text{is XML}}\ (\text{X-1}) \qquad \frac{}{\texttt{<T>}\ \text{is XML}}\ (\text{X-2}) \qquad \frac{X\ \text{is XML}}{\texttt{<T>}X\texttt{</T>}\ \text{is XML}}\ (\text{X-3}) \qquad \frac{X\ \text{is XML} \qquad Y\ \text{is XML}}{XY\ \text{is XML}}\ (\text{X-4})$$

- is XML: <xml>, <xml></xml>, <xml><a></a><c></xml>

- not XML: <a>Hi!</a>

$$\frac{\dfrac{\dfrac{}{\text{is XML}}^{(X-1)}}{\texttt{<a></a>}\ \text{is XML}}^{(X-3)} \qquad \dfrac{\dfrac{}{\texttt{<c>}\ \text{is XML}}^{(X-2)}}{\texttt{<b><c></b>}\ \text{is XML}}^{(X-3)}}{\texttt{<a></a><b><c></b>}\ \text{is XML}}^{(X-4)}}{\texttt{<xml><a></a><b><c></b></xml>}\ \text{is XML}}\ (\text{X-3})$$

# Inference rules can have other judgments as premises

$$\frac{n \text{ is a natural number}}{n \geq n}\text{(GE-EQ)}$$

$$\frac{m \text{ is a natural number} \qquad m \geq n}{m+1 \geq n}\text{(GE-+)}$$

Note: Using the same metavariable twice (e.g., n in GE-EQ) means *the same* expression needs to get substituted for both!

$$\dfrac{\dfrac{\dfrac{\dfrac{}{0 \text{ is a nat}}}{1 \text{ is a nat}}}{2 \text{ is a nat}} \qquad \dfrac{\dfrac{\dfrac{}{0 \text{ is a nat}}}{1 \text{ is a nat}}}{2 \text{ is a nat}} \qquad \dfrac{\dfrac{\dfrac{}{0 \text{ is a nat}}}{1 \text{ is a nat}} \qquad \dfrac{\dfrac{}{0 \text{ is a nat}}}{0 \geq 0}(GE-EQ)}{1 \geq 0}(GE+)}{2 \geq 0}(GE+)}{3 \geq 0}(GE+)$$

# We can prove things about *all possible derivations* by induction

- If n is a natural number, n >= 0.
  - How do we know?
  - If n is a natural number, then there is a derivation *saying* it's a natural number
  - That derivation is a finite object built from the inference rules we know
  - We can do induction over the *structure* of the derivation.

- That's all I'm going to say about that in this class.
  - If you want to learn (way, way) more, take CSE 5095: Types and Programming Languages!

# We don’t (usually) prove things about full programming languages

```
typexpr        ::= ' ident
                 | _
                 | ( typexpr )
                 | [[?]label-name:] typexpr -> typexpr
                 | typexpr { * typexpr }*
                 | typeconstr
                 | typexpr typeconstr
                 | ( typexpr { , typexpr } ) typeconstr
                 | typexpr as ' ident
                 | polymorphic-variant-type
                 | < [..] >
                 | < method-type { ; method-type } [; | ; ..] >
                 | # classtype-path
                 | typexpr # class-path
                 | ( typexpr { , typexpr } ) # class-path

poly-typexpr ::= typexpr
               | { ' ident }* . typexpr

method-type ::= method-name : poly-typexpr

argument       ::= expr
                 | ~ label-name
                 | ~ label-name : expr
                 | ? label-name
                 | ? label-name : expr

pattern-matching ::= [ | ] pattern [when expr] -> expr { | pattern [when expr] -> expr }

let-binding    ::= pattern = expr
                 | value-name { parameter } [: typexpr] [:> typexpr] = expr
                 | value-name : poly-typexpr = expr

parameter      ::= pattern
                 | ~ label-name
                 | ~ ( label-name [: typexpr] )
                 | ~ label-name : pattern
                 | ? label-name
                 | ? ( label-name [: typexpr] [= expr] )
                 | ? label-name : pattern
                 | ? label-name : ( pattern [: typexpr] [= expr] )
```

```
constant ::= integer-literal
           | int32-literal
           | int64-literal
           | nativeint-literal
           | float-literal
           | char-literal
           | string-literal
           | constr
           | false
           | true
           | ()
           | begin end
           | []
           | [||]
           | `tag-name

local-open     ::=
                 | let open module-path in expr
                 | module-path .( expr )
                 | module-path .[ expr ]
                 | module-path .[| expr |]
                 | module-path .{ expr }
                 | module-path .{< expr >}

object-expr    ::=
                 | new class-path
                 | object class-body end
                 | expr # method-name
                 | inst-var-name
                 | inst-var-name <- expr
                 | {< [ inst-var-name [= expr] { ; inst-var-name [= expr] } [;] ] >}
```

```
pattern ::= value-name
          | _
          | constant
          | pattern as value-name
          | ( pattern )
          | ( pattern : typexpr )
          | pattern | pattern
          | constr pattern
          | `tag-name pattern
          | #typeconstr
          | pattern { , pattern }*
          | { field [: typexpr] [= pattern]{ ; field [: typexpr] [= pattern] } [; _ ] [ ; ] }
          | [ pattern { ; pattern } [ ; ] ]
          | pattern :: pattern
          | [| pattern { ; pattern } [ ; ] |]
          | char-literal .. char-literal
          | lazy pattern
          | exception pattern
          | module-path .( pattern )
          | module-path .[ pattern ]
          | module-path .[| pattern |]
          | module-path .{ pattern }

type-definition    ::= type [nonrec] typedef { and typedef }

typedef            ::= [type-params] typeconstr-name type-information

type-information   ::= [type-equation] [type-representation] { type-constraint }

type-equation      ::= = typexpr

type-representation ::= = [|] constr-decl { | constr-decl }
                     | = record-decl
                     | = |

type-params        ::= type-param
                     | ( type-param { , type-param } )

type-param         ::= [ext-variance] ' ident

ext-variance       ::= variance [injectivity]
                     | injectivity [variance]
```

```
expr ::= value-path
       | constant
       | ( expr )
       | begin expr end
       | ( expr : typexpr )
       | expr { , expr }
       | constr expr
       | `tag-name expr
       | expr :: expr
       | [ expr { ; expr } [;] ]
       | [| expr { ; expr } [;] |]
       | { field [: typexpr] [= expr]{ ; field
       | { expr with field [: typexpr] [= expr]{
       | expr { argument }*
       | prefix-symbol
       | ~
       | -. expr
       | expr infix-op
       | expr . field
       | expr . field <- expr
       | expr .( expr )
       | expr .( expr ) <- expr
       | expr .[ expr ]
       | expr .[ expr ] <- expr
       | if expr then expr [ el
       | while expr do exp
       | for value-nam
       | expr ; expr
       | match expr with pattern-matching
       | function pattern-matching
       | fun { parameter }* [ : typexpr ] -> expr
       | try expr with pattern-matching
       | let [rec] let-binding { and let-binding } in exp
       | let exception constr-decl in expr
       | let module module-name { ( module-name : module-type ) } [ : module-type
         module-expr in expr
       | ( expr :> typexpr )
       | ( expr : typexpr :> typexpr )
       | assert expr
       | lazy expr
       | local-open
       | object-expr
```

```
variance       ::= +
                 | -

injectivity    ::= !

record-decl    ::= { field-decl { ; field-decl } [;] }

constr-decl    ::= (constr-name | [] | (::)) [ of constr-args ]

constr-args    ::= typexpr { * typexpr }

field-decl     ::= [mutable] field-name : poly-typexpr

type-constraint ::= constraint typexpr = typexpr
```

TLSFDR (Too long and in small font; didn’t read): It’s big!

# Instead, we prove things about small models of languages (“calculi”/“core calculi”)

- Most of the other details of languages don’t add much to the proofs anyway

Chapter 3

The Essence of ALGOL

*John C. Reynolds*

The Esser

Jérôme Siméon
Bell Laboratories

Featherweight Go

ROBERT GRIESEMER, Google, USA
RAYMOND HU, University of Hertfordshire, UK
WEN KOKKE, University of Edinburgh, UK
JULIEN LANGE, Royal Holloway, University
IAN LANCE TAYLOR, Google, USA
BERNARDO TONINHO, NOVA-LINCS, FC
PHILIP WADLER, University of Edinburgh,
NOBUKO YOSHIDA, Imperial College Lon

Featherweight Java: A Minimal Core Calculus for Java and GJ

ATSUSHI IGARASHI
University of Tokyo
BENJAMIN C. PIERCE
University of Pennsylvania
and
PHILIP WADLER
Avaya Labs

Edit Search

Save Search

Expand your search to The ACM Guide to Computing Literature (3,973,140 records)

Featherweight X10: A Core Calculus for Async-Finish Parallelism

Jonathan K. Lee Jens Palsberg
UCLA, University of California, Los Angeles
{jkenl,palsberg}@cs.ucla.edu

# Today (through spring break): the core calculus that started it all

- But first a history lesson…

# Hilbert's Three Questions

David Hilbert

Kurt Gödel

Is mathematics…

1. Complete?
   For any P, can we prove either P or ¬P?

   **No**

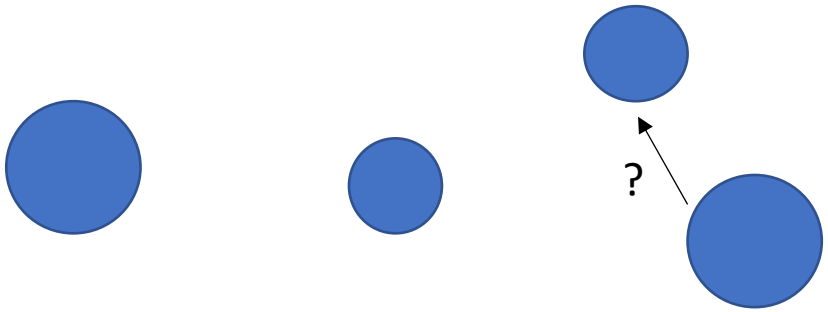
2. Consistent?
   Are we sure we can't prove False?

   **No***
   **(can't prove it within same system)**

3. Decidable?
   Is there an algorithm that, for any P, can decide whether P is ~~true~~? provable?
   *Entscheidungsproblem*

# Solving the *Entscheidungsproblem*

- Need a formal model of “any computation”



# Formalizing computation was nothing new

- Muḥammad ibn Mūsā al-Khwārizmī (780-850)
  Numerous *algorithms* in arithmetic, algebra

- Augusta Ada King, Countess of Lovelace (1815-1852)
  Notes about a universal
  computing machine

# The Untyped Lambda Calculus, 1935

Can model all “computable functions” with lambda functions

There is no computable function that can determine if two lambdas are equivalent.

Alonzo Church

# Hilbert's Three Questions

Is mathematics…

1. Complete?
   For any P, can we prove either P or ¬P?

   **No**

2. Consistent?
   Are we sure we can't prove False?

   **No**

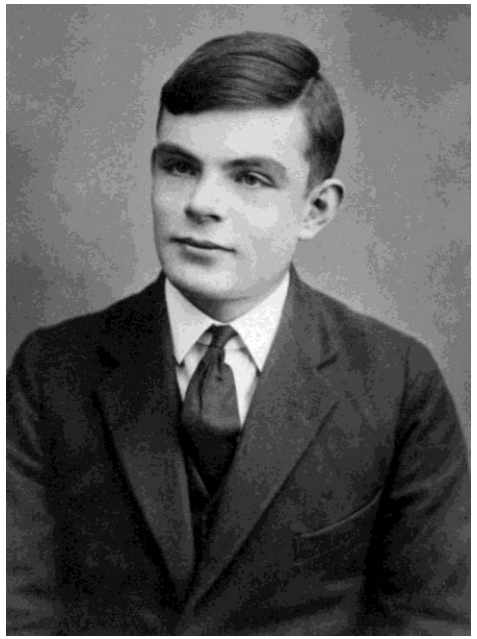
3. Decidable?
   Is there an algorithm that, for any P, can decide whether P is ~~true~~? provable?
   *Entscheidungsproblem*

   **No**

# Automatic Machines (a-machines), 1936

Can model all computable functions with a finite state machine with an infinite, writable “tape”.

There is no a-machine that can determine whether another a-machine halts.



Alan Turing

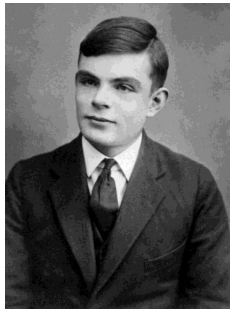
230 A. M. TURING [Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

*By* A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

# Hilbert's Three Questions



Is mathematics…

1. Complete?
   For any P, can we prove either P or ¬P?

   **No**

2. Consistent?
   Are we sure we can't prove False?

   **No**

3. Decidable?
   Is there an algorithm that, for any P, can decide whether P is ~~true~~? provable?
   *Entscheidungsproblem*

   **No** **No**

# Today: the syntax of the (untyped) lambda calculus

- We specify syntax in Backus-Naur Form (BNF)

A ::= B | C | A A

one of these things (which can be “terminals”, i.e., actual syntactic things, or other nonterminals)

A construct (“nonterminal”) A

is defined to be

# Example: arithmetic expressions

e ::= n | e + e | e – e | e * e

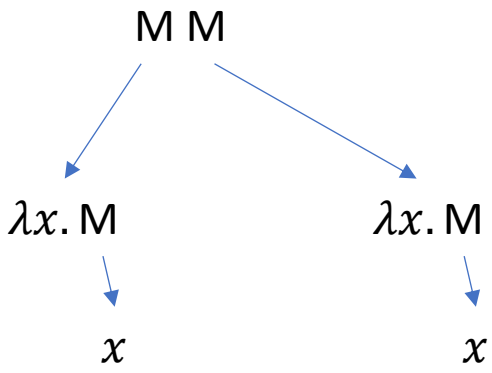
# Syntax of the untyped lambda calculus

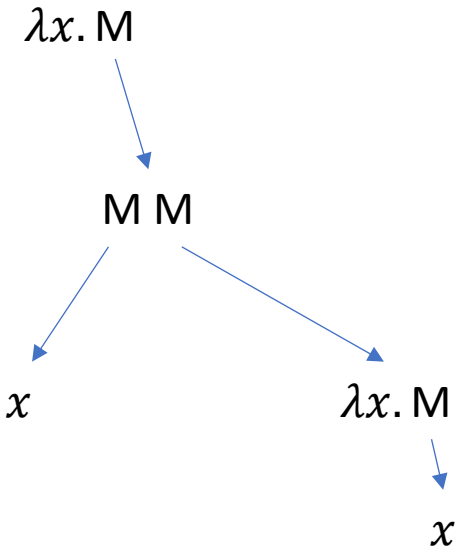
- Terms M ::= x | λx. M | M M

Variables | One-argument anonymous functions | Application

# BNF form specifies “Abstract syntax” (as opposed to “concrete syntax”)

- Concrete syntax: $(\lambda x.\, x)\ (\lambda x.\, x)$ vs. $\lambda x.\, x\ \lambda x.\, x$
- Abstract syntax
  - (Abstract syntax trees/ASTs)




(Concrete syntax rule: lambda bodies go as far right as possible until they hit a close-paren)

# We've already seen some important patterns in the lambda calculus

- Identity function $\lambda x.\, x$
- Currying $\lambda x.\, \lambda y.\, y\ x$
- Higher-order functions $\lambda f.\, f\ M$
- Eta equivalence $\mathrm{M} \equiv \lambda x.\, M\ x$ (assuming x doesn't appear free in M)